Material de Correção – Prova 01

Disciplina: Estruturas de Dados II
Prof. Tiago Pessoa Ferreira de Lima

Conteúdo

1 Questão 1: Média da Árvore **1**

2 Questão 2: Caminho de Soma Máxima **2**

3 Questão 3: Esvaziar Árvore (Pós-Ordem) **2**

4 Questão 4: Árvore Simétrica (Espelho) **3**

---

1 Questão 1: Média da Árvore

Conteúdo Relacionado: *Percursos em Árvores (DFS) e Acumuladores.*

Enunciado: Implemente o método `media(self)` na classe `ArvoreBinaria` que retorna a média aritmética de todos os valores armazenados. Caso a árvore esteja vazia, retorne `None`.

Fundamentação Teórica: Esta questão aplica o conceito de **Caminhamento em Árvore**. Para calcular a média, é obrigatório visitar cada nó exatamente uma vez ($O(n)$). A estratégia utiliza uma função recursiva que retorna múltiplos valores (soma e contagem), demonstrando como a recursão pode "subir"informações das subárvores para a raiz.

Implementação:

```python
def media(self):
    if self.raiz is None:
        return None

    def dfs_soma_qtd(no):
        if no is None:
            return 0, 0 # Base da recurs o: (soma, quantidade)

        soma_esq, qtd_esq = dfs_soma_qtd(no.esquerda)
        soma_dir, qtd_dir = dfs_soma_qtd(no.direita)

        soma_total = no.valor + soma_esq + soma_dir
        qtd_total = 1 + qtd_esq + qtd_dir

        return soma_total, qtd_total

    soma, qtd = dfs_soma_qtd(self.raiz)
    return soma / qtd
```

Complexidade: $O(n)$ em tempo e $O(h)$ em memória.

---

2 Questão 2: Caminho de Soma Máxima

Conteúdo Relacionado: *Busca em Profundidade (DFS) e Backtracking.*

Enunciado: Retorne uma lista com os valores do caminho da raiz até uma folha cuja soma seja a maior possível.

Fundamentação Teórica: Aqui exploramos o conceito de **Backtracking**. Diferente de apenas somar valores, precisamos manter o rastro do caminho atual. Ao "voltar"na recursão (`pop`), limpamos o estado para explorar novas ramificações. É um problema clássico de otimização em estruturas hierárquicas.

Implementação:

```python
def caminho_soma_maxima(self):
    if self.raiz is None:
        return []

    maior_soma = float('-inf')
    melhor_caminho = []

    def dfs(no, caminho_atual, soma_atual):
        nonlocal maior_soma, melhor_caminho
        if no is None: return

        caminho_atual.append(no.valor)
        soma_atual += no.valor

        if no.esquerda is None and no.direita is None: # N   Folha
            if soma_atual > maior_soma:
                maior_soma = soma_atual
                melhor_caminho = caminho_atual.copy()
        else:
            dfs(no.esquerda, caminho_atual, soma_atual)
            dfs(no.direita, caminho_atual, soma_atual)

        caminho_atual.pop() # Passo crucial de Backtracking

    dfs(self.raiz, [], 0)
    return melhor_caminho
```

---

3 Questão 3: Esvaziar Árvore (Pós-Ordem)

Conteúdo Relacionado: *Percurso Pós-Ordem (LRN) e Gerenciamento de Memória.*

Enunciado: Remova todos os nós da árvore. Os filhos devem ser processados antes dos pais.

Fundamentação Teórica: O uso do **Percurso Pós-Ordem** é fundamental aqui. Em sistemas de baixo nível (como C++), se você apagar o pai antes dos filhos, perde o acesso (ponteiro) para os filhos, gerando vazamento de memória. Mesmo em Python, seguir a ordem *Esquerda* $\rightarrow$ *Direita* $\rightarrow$ *Raiz* garante uma limpeza lógica consistente com a estrutura de dependência da árvore.

Implementação:

```python
def esvaziar_arvore(self):
    def pos_ordem_limpar(no):
        if no is None: return

```

```
        pos_ordem_limpar(no.esquerda) # L
        pos_ordem_limpar(no.direita)  # R
        no.esquerda = None            # N (Visita/Limpeza)
        no.direita = None

    pos_ordem_limpar(self.raiz)
    self.raiz = None
```

---

4 Questão 4: Árvore Simétrica (Espelho)

Conteúdo Relacionado: *Recursão e Propriedades Estruturais de Árvores.*

Enunciado: Verifique se a árvore é um espelho de si mesma.

Fundamentação Teórica: Esta questão exercita a **Recursão em Duas Árvores** simultaneamente. A simetria não é apenas sobre valores iguais, mas sobre posições invertidas: o filho esquerdo do lado A deve ser igual ao filho direito do lado B. É uma aplicação prática da definição recursiva de igualdade estrutural.

Implementação:

```
def eh_simetrica(self):
    if self.raiz is None: return True

    def espelho(no1, no2):
        if not no1 and not no2: return True
        if not no1 or not no2: return False

        return (no1.valor == no2.valor and
                espelho(no1.esquerda, no2.direita) and
                espelho(no1.direita, no2.esquerda))

    return espelho(self.raiz.esquerda, self.raiz.direita)
```